

# Pytania z Odpowiedziami

## Spis Treści

- Rozdział 1 - Wprowadzenie
- Rozdział 2 - Typy danych, instrukcje i wyrażenia
- Rozdział 3 - Funkcje
- Rozdział 4 - Operacje na liczbach
- Rozdział 5 - Operacje na znakach i łańcuchach znaków
- Rozdział 6 - Operacje na kolekcjach
- Rozdział 7 - Obsługa wyjątków
- Rozdział 8 - Operacje na plikach
- Rozdział 9 - Tworzenie graficznego interfejsu użytkownika (GUI)
- Rozdział 10 - Inne

Poprawne odpowiedzi zaznaczone są kolorem zielonym w **taki** lub w **taki** sposób.

## Rozdział 1 - Wprowadzenie

### Pytanie 1

Python jest językiem:

- A. Statycznie typowanym
- B. **Dynamicznie typowanym**
- C. Odpowiedzi A i B są prawidłowe
- D. Żadne z powyższych

### Pytanie 2

Co oznacza skrót *PEP*?

- A. **Python Enhancement Proposals**
- B. *Python Enhancement Plans*
- C. *Python Environmental Programming*
- D. *Python Eye Programming*

### Pytanie 3

Czym jest *PEP8*?

- A. To jeden z pakietów występujących w standardowej bibliotece języka Python

- B. To zbiór ośmiu, najważniejszych pakietów ze standardowej biblioteki języka Python
- C. **To dokument opisujący standardy dotyczące stylu kodowania w języku Python**
- D. To pseudonim jednego z głównych programistów odpowiedzialnych za rozwijanie języka Python

#### Pytanie 4

Czym jest *PATH*?

- A. **Jest to zmienna środowiskowa określająca zestaw katalogów, w których znajdują się programy wykonywalne**
- B. Jest to ścieżka do interpretera języka Python
- C. Jest to jeden z pakietów w standardowej bibliotece języka Python
- D. Jest to skrót od *Python Access Terminal Hierarchy*

#### Pytanie 5

Czym jest interpreter języka Python?

- A. Jest to stanowisko w firmie, w ramach którego tłumaczy się kod źródłowy na język biznesowy
- B. Jest to narzędzie środowiska PyCharm, które tłumaczy kod źródłowy języka Python na język ludzki
- C. Jest to program odpowiedzialny za tłumaczenie kodu źródłowego napisanego w języku Python na kod maszynowy
- D. **Jest to program odpowiedzialny za wykonywanie kodu źródłowego napisanego w języku Python**

## Rozdział 2 - Typy danych, instrukcje i wyrażenia

#### Pytanie 1

Programista chce przypisać cyfrę `2` do trzech zmiennych `a`, `b`, `c` w jednej linii kodu. Które z przedstawionych rozwiązań jest poprawne?

- A. `a, b, c = 2`
- B. `a = b = c = 2`
- C. `a, b, c = 2, 2, 2`
- D. **Odpowiedzi B i C są poprawne**

#### Pytanie 2

Programista chce przeiterować po kluczach i wartościach słownika z wykorzystaniem pętli `for`. Które z przedstawionych rozwiązań jest poprawne?

- A. `for (key, value) in some_dict.items()`

- B. `for (key, value) in some_dict`
- C. `for pair in some_dict`
- D. Żadne z powyższych

### Pytanie 3

Dana jest lista `numbers = [1, 2, 3, 4, 5]`. Programista chce przypisać pierwszy element listy do zmiennej `a`, ostatni do zmiennej `c`, natomiast środkowe trzy w postaci listy do zmiennej `b`. Które z przedstawionych rozwiązań jest poprawne?

- A. `a, b, c = numbers`
- B. `a, *b, c = numbers`
- C. `a_first, c_last, b_middle = numbers`
- D. `a, _, c = numbers`

### Pytanie 4

Wybierz poprawny zapis operacji przypisania z aktualizacją dla mnożenia:

- A. `number = number * 1`
- B. `number == number * 1`
- C. `number *= 1`
- D. `number.update(1)`

### Pytanie 5

Czy kod spod instrukcji `if not True and False` zostanie wykonany?

- A. Tak
- B. Nie

### Pytanie 6

Dane są zmienne `x = 1` i `y = 2`. Czy kod spod instrukcji `if x == y or True` zostanie wykonany?

- A. Tak
- B. Nie

### Pytanie 7

Dane są zmienne `x = 1` i `y = 2`. Co zostanie wyświetlone w konsoli po wywołaniu funkcji `print(x is y)`?

- A. `True`
- B. `False`
- C. `None`

D. [1, 2]

### Pytanie 8

Dane są zmienne `a, b, c = 1, 2, 3`. Jaki będzie wynik zdania logicznego `((a > b) and (c < a)) or (not (b == a) and (c <= b))` ?

- A. True
- B. False

### Pytanie 9

Wybierz poprawną formę operatora trójargumentowego:

- A. `result = value_if_true if condition else value_if_false`
- B. `result = value_if_false if condition else value_if_true`
- C. Odpowiedzi A i B są prawidłowe
- D. Żadne z powyższych

### Pytanie 10

Dana jest zmienna `x = 13`. Jaką wartość przyjmie zmienna `result` dla kodu `result = "Even" if x % 2 == 0 else "Odd"` ?

- A. True
- B. False
- C. Even
- D. Odd

### Pytanie 11

Dana jest zmienna `x = 7`. Czy zostanie wykonany kod spod instrukcji `if isinstance(x, (int, float))` ?

- A. Tak
- B. Nie

### Pytanie 12

Dany jest kod źródłowy:

```
x = 1

match x:
    case 0:
        print("x = 0")
    case 1:
        print("x = 1")
```

Co zostanie wyświetlone w konsoli po jego wykonaniu?

- A. `x = 0`
- B. `x = 1`
- C. `True`
- D. `False`

### Pytanie 13

Od kiedy jest liczony czas *Unixowy*?

- A. Od 1 stycznia 2000 roku
- B. **Od 1 stycznia 1970 roku**
- C. Od 1 stycznia 1900 roku
- D. Od powstania pierwszej dystrybucji systemu Linux

### Pytanie 14

Co to jest UTC?

- A. **Jest to skrót od tak zwanego uniwersalnego czasu koordynowanego i odnosi się do czasu na długości geograficznej zero stopni**
- B. Jest to skrót od nazwy strefy czasowej obowiązującej w Stanach Zjednoczonych Ameryki
- C. Jest to główny pakiet standardowej biblioteki języka Python do pracy z datami i czasem
- D. Żadne z powyższych

### Pytanie 15

Dany jest kod źródłowy:

```
from datetime import datetime

# Input date in the format "yyyy-mm-dd"
input_date = "2020-01-12"

# Convert the input date to a datetime object
datetime_obj = datetime.strptime(input_date, "%Y-%m-%d")
```

Programista chce przekonwertować datę `2020-01-12` do formatu `12-01-2020`. Które z podanych rozwiązań jest poprawne?

- A. `output_date = datetime_obj.strftime("%d.%m.%Y")`
- B. `output_date = datetime_obj.strftime(input_date)`
- C. `output_date = datetime_obj.strftime("12.01.2020")`
- D. Odpowiedzi A i C są poprawne

Choć kod z odpowiedzi C również przypisze do zmiennej `output_date` wartość `12.01.2020`,

to pytanie dotyczy konwersji daty wyrażonej w jednym formacie na inny. Biorąc to pod uwagę jest tylko jedna poprawna odpowiedź, a mianowicie odpowiedź A.

## Rozdział 3 - Funkcje

### Pytanie 1

Jaka jest poprawna forma funkcji anonimowej korzystającej z wyrażenia `lambda` ?

- A. `lambda parameters: expression`
- B. `lambda expression parameters`
- C. `lambda(parameters): expression`
- D. `lambda parameters -> expression`

### Pytanie 2

Jaka jest poprawna forma funkcji anonimowej obliczającej sumę kwadratów liczb?

- A. `lambda nums: sum([num**2 for num in nums])`
- B. `lambda sum([num**2 for num in nums])`
- C. `lambda nums: num**2 for num in nums`
- D. Żadne z powyższych

### Pytanie 3

Czym jest rekurencja?

- A. To sposób na ukrycie implementacji algorytmu przed innymi programistami.
- B. To technika optymalizacji wydajności kodu w językach programowania.
- C. To metoda unikania używania pętli w kodzie i zastępowania ich przez wywołania funkcji, co sprawia, że program jest bardziej wydajny i szybszy.
- D. **To koncepcja programowania, w której funkcja wywołuje samą siebie bezpośrednio lub pośrednio w czasie swojego działania.**

### Pytanie 4

Dany jest kod źródłowy:

```
def sum(x):
    if not x:
        return 0
    else:
        return x[0] + sum(x[1:])

numbers = [1, 2, 3, 4, 5]
s = sum(numbers)
print(s)
```

Co zostanie wyświetlone w konsoli w wyniku jego wykonania?

- A. `[1, 2, 3, 4, 5]`
- B. `[1, 3, 6, 10, 15]`
- C. `15`
- D. `0`

### Pytanie 5

Czym jest dekorator funkcji?

- A. **To funkcja, która przyjmuje inną funkcję jako argument i dodaje do niej funkcjonalność lub modyfikuje jej zachowanie, bez konieczności zmiany samej funkcji.**
- B. To specjalne komentarze, które mogą być wykorzystywane jedynie w celu dokumentowania funkcji.
- C. To syntaktyczne uproszczenie definicji funkcji, które pozwala korzystać z tradycyjnych funkcji tak, jak z funkcji anonimowych.
- D. To technika rozbudowywania funkcjonalności funkcji w definicji tej funkcji.

## Rozdział 4 - Operacje na liczbach

### Pytanie 1

Dana jest liczba `pi = 3.14159`. Wybierz kod, który pozwoli zaokrąglić ją do dwóch miejsc po przecinku:

- A. `decimal(pi, round=2)`
- B. `pi.round(2)`
- C. `round(pi, 2)`
- D. `float(pi, 2)`

### Pytanie 2

Dana jest liczba `pi = 3.14159`. Wybierz kod, który wykorzystując `f-String` pozwoli na wyświetlenie w konsoli `Pi is equal to 3.14159`:

- A. `print("Pi is equal to {pi}")`
- B. `print(f"Pi is equal to {pi}")`
- C. `print("Pi is equal to %pi")`
- D. `print("Pi is equal to $pi")`

### Pytanie 3

Dana jest liczba `pi = 3.14159`. Wybierz kod, który wykorzystując formatowanie liczb zmiennoprzecinkowych pozwoli na wyświetlenie w konsoli liczby `pi` zaokrąglonej do dwóch

miejsc po przecinku:

- A. `print("{pi:round.2f}")`
- B. `print(f"{pi}:.2f")`
- C. `print(f"{pi:.2f}")`
- D. `print(f"{format(pi:.2f)}")`

#### Pytanie 4

Dana jest liczba `number = 0.25` . Wybierz kod, który wykorzystując formatowanie liczb zmiennoprzecinkowych pozwoli na wyświetlenie w konsoli `The increase was 25%` :

- A. `print(f"The increase was {number:.%}")`
- B. `print(f"The increase was {number:.0%}")`
- C. `print(f"The increase was {number:.1%}")`
- D. `print(f"The increase was {number:f%}")`

#### Pytanie 5

Dany jest kod:

```
import math

a, b = 50.0, 41.0
is_close = math.isclose(a, b, rel_tol=0.2)

print(is_close)
```

Co zostanie wyświetlone w konsoli?

- A. `True`
- B. `False`
- C. `50.0`
- D. `41.0`

#### Pytanie 6

Które z wymienionych funkcji należą do modułu `statistics` ?

- A. `mean, fmean, median, mode`
- B. `mean, median, dominante, var`
- C. `fmean, median, mode, variance`
- D. **Odpowiedzi A i C są prawidłowe**

#### Pytanie 7

Dany jest kod:



```
import random

a, b = -5, 5
```

Które z poniższych rozwiązań umożliwia wygenerowanie liczby z zakresu między -5 a 5?

- A. `random.range(a, b)`
- B. `random.uniform(a, b)`
- C. `random.random(a, b)`
- D. `random.generator(a, b)`

## Rozdział 5 - Operacje na znakach i łańcuchach znaków

### Pytanie 1

Które z poniższych rozwiązań umożliwia pobranie pierwszej litery `i` występującej w łańcuchu znaków `name = "Marcin Mikiciuk"` ?

- A. `letter = name[4]`
- B. `letter = name[5]`
- C. `letter = name['i']`
- D. `letter = name.find('i')`

### Pytanie 2

Dany jest kod źródłowy:

```
name = "Marcin Mikiciuk"
as_list = list(name)
print(as_list[:6])
```

Co zostanie wyświetlone w konsoli po jego wykonaniu?

- A.
- B. `n`
- C. `Marcin`
- D. `['M', 'a', 'r', 'c', 'i', 'n']`

### Pytanie 3

Dany jest kod źródłowy:

```
numbers = [1, 2, 3]
as_string = '#'.join(numbers)
print(as_string)
```

Co zostanie wyświetlone w konsoli po jego wykonaniu?

- A. 1#2#3
- B. #123
- C. [1, '#', 2, '#', 3]
- D. **TypeError: sequence item 0: expected str instance, int found**

#### Pytanie 4

Dany jest kod źródłowy:

```
x = "Marcin Mikiciuk"
...
print(x)
```

Wybierz rozwiązanie, które należy wstawić w miejsce `...`, aby po wykonaniu kodu został wyświetlony w konsoli łańcuch znaków `Pan Marcin Mikiciuk nie może teraz odebrać telefonu. .`

- A. `x = f'Pan {x} nie może teraz odebrać telefonu.'`
- B. `x = x.join(['Pan ', ' nie może teraz odebrać telefonu.'])`
- C. `x = 'Pan {x} nie może teraz odebrać telefonu.'.format(x=x)`
- D. **Wszystkie powyższe odpowiedzi są poprawne**

#### Pytanie 5

Dany jest łańcuch znaków `name = "marcin"`. Które z poniższych rozwiązań pozwoli zamienić znak `m` na `M`?

- A. `name.upper()`
- B. **`name.capitalize()`**
- C. `name.cap(0)`
- D. Żadne z powyższych

#### Pytanie 6

Dany jest kod źródłowy:

```
import re

sentence = "Pan Marcin przelał już pieniądze."
```

Które z poniższych rozwiązań zwróci po wykonaniu krotkę `(4, 10)`?

- A. `re.span('Marcin', sentence)`
- B. `sentence.search('Marcin')`

- C. `re.search('Marcin', sentence)`
- D. `re.search('Marcin', sentence).span()`

### Pytanie 7

Dany jest kod źródłowy:

```
from string import Template

name, age = 'Marcin', 65

template = Template("My name is $n and I am $a years old.")
formatted_string = template.([???])(n=name, a=age)

print(formatted_string)
```

Jaką funkcję należy wstawić w miejsce `[???]`, aby po uruchomieniu kodu w konsoli zostało wyświetlone `My name is Marcin and I am 65 years old.` ?

- A. `build`
- B. `set`
- C. `substitute`
- D. `construct`

### Pytanie 8

Co oznacza skrót *regex*?

- A. *regular expression*
- B. *regular exercise*
- C. *regular strings*
- D. *regular generative string formatting*

### Pytanie 9

Dany jest kod źródłowy:

```
import re

string = "I have 51 apples and 35 oranges."
formatted_string = re.sub([???], 'ZERO', string)

print(formatted_string)
```

Co należy wstawić w miejsce `[???]`, aby po wykonaniu kodu w konsoli zostało wyświetlone zdanie `I have ZERO apples and ZERO oranges.` ?

- A. `r'\d'`
- B. `f'\d+'`
- C. `r'\d'`
- D. `r'\d+'`

## Pytanie 10

Dany jest kod źródłowy:

```
import re

string = "  To zdanie  zawiera zdecydowanie  zbyt wiele  spacji!  "
cleaned_string = re.sub([???], [???], string.strip())

print(cleaned_string)
```

Co należy wstawić w miejsca `[???]` , aby po wykonaniu kodu w konsoli zostało wyświetlone zdanie `To zdanie zawiera zdecydowanie zbyt wiele spacji!` ?

- A. `f'\s+` oraz `' '`
- B. `r'\s+'` oraz `\\s`
- C. `r'\s+'` oraz `' '`
- D. `f'\s+` oraz `\\s`

## Rozdział 6 - Operacje na kolekcjach

### Pytanie 1

Wybierz zdanie, które jest prawdziwe:

- A. Iterator w Pythonie służy wyłącznie do iteracji po liczbach całkowitych.
- B. Iterator jest jednorazowym obiektem i nie można go używać wielokrotnie.
- C. Każda kolekcja w Pythonie jest automatycznie iteratorem i nie wymaga dodatkowego zastosowania.
- D. **Iterator w języku Python to obiekt, który umożliwia iterację po elementach kolekcji (np. listy, krotki) lub sekwencji danych.**

### Pytanie 2

Wybierz zdanie, które jest prawdziwe:

- A. Generator w Pythonie to wbudowany typ danych do przechowywania sekwencji wartości.
- B. **Generator w języku Python to funkcja zawierająca przynajmniej jedno wyrażenie `yield` , które służy do generowania kolejnych wartości w trakcie iteracji.**
- C. Generator jest jednorazowym obiektem i nie można go używać wielokrotnie.
- D. Generatory w Pythonie nie mogą być używane w pętlach `for` do iteracji po kolekcjach.

### Pytanie 3

Dany jest kod źródłowy:

```
def fibonacci():  
    a, b = 0, 1  
    while True:  
        yield a  
        a, b = b, a + b  
  
fib_gen = fibonacci()  
  
print(next(fib_gen))
```

Co zostanie wyświetlone w konsoli po jego uruchomieniu?

- A. <generator object fibonacci at 0x000001B29B419630>
- B. 0
- C. 1
- D. TypeError: next expected at least 2 arguments, got 1

### Pytanie 4

Które z poniższych rozwiązań pozwala na stworzenie następującej listy: [1, 3, 5, 7, 9] ?

- A. [x for x in range(1, 11, 2)]
- B. [x for x in range(1, 11)]
- C. [x for x in range(1, 9, 2)]
- D. [x for x in range(1, 9)]

### Pytanie 5

Korzystając z moduły `string` oraz techniki składania słowników (*dictionary comprehension*) stwórz słownik, którego kluczami są małe litery alfabetu łacińskiego, a wartościami liczby od 1 do 26.

**Kod źródłowy:**

```
import string  
my_dict = {char: ord(char) - ord('a') + 1 for char in string.ascii_lowercase}  
print(my_dict)
```

**Wynik:**

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6, 'g': 7, 'h': 8, 'i': 9, 'j': 10, 'k': 11, 'l': 12, 'm': 13, 'n': 14, 'o': 15, 'p': 16, 'q': 17, 'r': 18, 's': 19, 't': 20, 'u': 21, 'v': 22, 'w': 23, 'x': 24, 'y': 25, 'z': 26}
```

## Pytanie 6

Dany jest kod źródłowy:

```
employees = ['Mikiciuk', 'Kowalski', 'Raczkowska', 'Lisicka']

for index, employee in enumerate(employees):
    print(index, employee, end=' ')
```

Jaki będzie wynik działania przedstawionego kodu?

- A. Mikiciuk Kowalski Raczkowska Lisicka
- B. 0 Mikiciuk 1 Kowalski 2 Raczkowska 3 Lisicka
- C. 1 Mikiciuk 2 Kowalski 3 Raczkowska 4 Lisicka
- D. enum(0) Mikiciuk enum(1) Kowalski enum(2) Raczkowska enum(3) Lisicka

## Pytanie 7

Dana jest lista `my_list = [3, 1, 4, 1, 5, 9]`. Które z poniższych rozwiązań pozwala posortować elementy listy rosnąco?

- A. `sorted_list = sorted(my_list)`
- B. `sorted_list = my_list.sorted()`
- C. `sorted_list = sort(my_list)`
- D. `sorted_list = my_list.sort()`

## Pytanie 8

Dany jest kod źródłowy:

```
first_names = ['Alice', 'Bob', 'Charlie']
last_names = ['Smith', 'Bronson', 'Sheen']
ages = [25, 30, 35]

zipped = zip(first_names, last_names, ages)

for fn, ln, a in zipped:
    print(fn, ln, a)
```

Co zostanie wyświetlone w konsoli w ramach drugiej iteracji pętli `for` ?

- A. 'Smith', 'Bronson', 'Sheen'
- B. Alice Smith 25
- C. Bob Bronson 30
- D. Charlie Sheen 35

## Pytanie 9

Dany jest kod źródłowy:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))
```

Co zostanie wyświetlone w konsoli podczas wykonania przedstawionego kodu?

- A. `<filter object at 0x00000122366A3250>` (wartość po `at` różni się przy każdym wykonaniu kodu)
- B. `[2, 4, 6, 8, 10]`
- C. `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`
- D. `None`

### Pytanie 10

Które rozwiązanie zaprezentowane poniżej pozwala stworzyć nową listę `new_list = [0, 0, 1, 1]` na podstawie istniejącej listy `numbers = [1, 2, 3, 4]` ?

- A. `new_list = map(numbers, above=2)`
- B. `new_list = map(lambda x: 1 if x >= 3 else 0, numbers)`
- C. `new_list = list(map(numbers, above=2))`
- D. `new_list = list(map(lambda x: 1 if x >= 3 else 0, numbers))`

### Pytanie 11

Dany jest kod źródłowy:

```
from functools import reduce

def multiply(x, y):
    return x * y

numbers = [1, 2, 3]

sum_of_numbers = reduce(multiply, numbers)
print(sum_of_numbers)
```

Co zostanie wyświetlone w konsoli po jego wykonaniu?

- A. `[1, 3, 6]`
- B. `6`
- C. `None`
- D. `<reduce object at 0x00000122366A3250>`

### Pytanie 12

Dany jest kod źródłowy:

```
from itertools import groupby

names = ['Piotr', 'Marcin', 'Andrzej', 'Paweł', 'Michał', 'Maciej']

[???]

for key, group in grouped_names:
    print(key, list(group))
```

Co należy wstawić w miejsce [???] , aby uzyskać następujący wynik działania skryptu w konsoli:

```
P ['Piotr']
M ['Marcin']
A ['Andrzej']
P ['Paweł']
M ['Michał', 'Maciej']
```

- A. `grouped_names = groupby(names)`
- B. `grouped_names = groupby(names, key=lambda x: x)`
- C. `grouped_names = groupby(names, key=lambda x: names)`
- D. `grouped_names = groupby(names, key=lambda x: x[0])`

### Pytanie 13

Dane są dwie krotki `t1 = (1, 2, 3)` and `t2 = (4, 5, 6)` . Które z poniższych rozwiązań pozwala połączyć obie krotki do jednej krotki `(1, 2, 3, 4, 5, 6)` ?

- A. `merge(t1, t2)`
- B. `concatenate(t1, t2)`
- C. `t1 + t2`
- D. `t1 * t2`

## Rozdział 7 - Obsługa wyjątków

### Pytanie 1

Które zdanie najlepiej opisuje obsługę wyjątków w języku Python?

- A. Obsługa wyjątków w Pythonie to sposób na ignorowanie błędów i kontynuowanie wykonywania programu bez żadnych konsekwencji.
- B. Obsługa wyjątków jest techniką używaną do celowego wprowadzania błędów do programu w celu przetestowania jego niezawodności.
- C. Obsługa wyjątków w Pythonie ma zastosowanie tylko do błędów składni i nie może



obsługiwać błędów uruchomieniowych.

- D. **Obsługa wyjątków w Pythonie pozwala na obsługę błędów i wyjątków, które występują podczas wykonywania programu, zapobiegając nagłemu zakończeniu programu.**

## Pytanie 2

Które instrukcje są wykorzystywane w obsłudze wyjątków?

- A. **try, except, else, finally**
- B. try, except, else, default
- C. try, except, default
- D. try, except, finally

## Pytanie 3

Jaka jest kolejność wykonywania instrukcji wykorzystywanych w obsłudze wyjątków?

- A. try, finally, except, else
- B. try, else, except, finally
- C. **try, except, else, finally**
- D. try, except, finally, else

## Pytanie 4

Dany jest kod źródłowy:

```
def validate_age(age):
    if age < 18:
        [??] ValueError("Age must be at least 18!")
    else:
        print("Age is valid.")

try:
    validate_age(10)
except ValueError as e:
    print(e)
```

Jakiej instrukcji brakuje w miejscu [??], aby wyjątek `ValueError` został podniesiony i odpowiednio obsłużony?

- A. call
- B. **raise**
- C. exception
- D. except

## Pytanie 5

W Pythonie słowo kluczowe `[???]` jest używane do przeprowadzania kontroli debugowania podczas programowania. Testuje ono warunek, który ma być prawdziwy. Jeśli warunek jest fałszywy, zgłasza błąd `AssertionError`, sygnalizując, że w kodzie może występować błąd lub problem logiczny.

Jakie słowo kluczowe powinno się znaleźć w miejscu `[???]` ?

- A. `raise`
- B. `except`
- C. `assert`
- D. `error`

## Rozdział 8 - Operacje na plikach

### Pytanie 1

Która funkcja z modułu `os` pozwala pobrać ścieżkę do aktualnej przestrzeni roboczej?

- A. `os.getcwd()`
- B. `os.cwd()`
- C. `os.getworkingpath()`
- D. `os.workingpath()`

### Pytanie 2

Która funkcja z modułu `os` pozwala stworzyć nowy folder w wybranej lokalizacji?

- A. `os.crdir()`
- B. `os.dir()`
- C. `os.createdir()`
- D. `os.makedirs()`

### Pytanie 3

Założmy, że chcemy utworzyć nowy plik w trybie zapisywania do niego treści. Od której z poniższych linii kodu zaczniemy?

- A. `with open('NewScript.txt', 'a') as file:`
- B. `with open('NewScript.txt', 'w') as file:`
- C. `with open('NewScript.txt', 'r') as file:`
- D. `with open('NewScript.txt') as file:`

## Rozdział 9 - Tworzenie graficznego interfejsu użytkownika (GUI)

## Pytanie 1

Czym jest *tkinter*?

- A. To pakiet wchodzący w skład standardowej biblioteki języka Python służący do tworzenia graficznego interfejsu użytkownika
- B. To pakiet służący do tworzenia graficznego interfejsu użytkownika, który nie wchodzi w skład standardowej biblioteki języka Python
- C. To jeden z pakietów standardowej biblioteki języka Python służący do pracy z czasem
- D. To jeden z pakietów standardowej biblioteki języka Python służący do realizowania zadań z zakresu programowania wielowątkowego i asynchronicznego

## Pytanie 2

Zakładając, że pierwsza linia skryptu, to `import tkinter as tk`, to które z poniższych rozwiązań pozwala utworzyć główne okno aplikacji korzystającej z pakietu *tkinter*?

- A. `root = tk.Window()`
- B. `root = tk.App()`
- C. `root = tk.Tk()`
- D. `root = tk.mainloop()`

## Pytanie 3

Która z funkcji pozwala uruchomić pętlę główną aplikacji korzystającej z pakietu *tkinter*?

- A. `main()`
- B. `mainloop()`
- C. `root()`
- D. `start()`

## Pytanie 4

Który z poniższych *widgetów* nie należy do głównego pakietu *tkinter*, ale należy za do podpakietu *ttk*?

- A. `Button`
- B. `Radiobutton`
- C. `Combobox`
- D. Odpowiedzi B i C są prawidłowe

## Pytanie 5

Która z poniższych funkcji pozwala wymusić początkowe rozmiary głównego okna aplikacji?

- A. `dimensions()`
- B. `widthandheight()`

- C. `size()`
- D. `geometry()`

### Pytanie 6

Trzy główne menedżery układu w pakiecie *tkinter*, to:

- A. *pack, place, grid*
- B. *pack, frame, grid*
- C. *frame, place, grid*
- D. *pack, frame, place*

### Pytanie 7

Dany jest kod źródłowy:

```
import tkinter as tk

root = tk.Tk()

def click():
    print("I'm Batman!")

button = tk.Button(text="Click Me!", [???])
button.pack()

root.mainloop()
```

Co należy wstawić w miejscu `[??]` , aby po uruchomieniu aplikacji naciśnięcie przycisku `button` powodowało wywołanie funkcji `click()` ?

- A. `action=click`
- B. `command=click`
- C. `run=click`
- D. `event_handle=click`

## Rozdział 10 - Inne

### Pytanie 1

Czym jest dokumentacja kodu źródłowego?

- A. **Zestaw pisemnych materiałów i komentarzy, które dostarczają informacji o programie, jego modułach, klasach i funkcjach**
- B. Proces sprawdzania i testowania kodu źródłowego w celu wyłapania w nim ewentualnych błędów i niedociągnięć
- C. Kompletny zbiór grafik i diagramów przedstawiających działanie programu, ułatwiający

wizualizację procesów programistycznych

- D. Sekretny kod lub tajny skrypt używany do ukrywania prawdziwych zamiarów programisty przed użytkownikami

## Pytanie 2

Czym jest debugowanie kodu źródłowego?

- A. Instrukcje lub polecenia umożliwiające wprowadzanie zamierzonego błędu w celu zrozumienia, jak program reaguje na różne sytuacje awaryjne
- B. Specjalny rodzaj kodu, który pozwala programiście śledzić działanie programu w czasie rzeczywistym i dostarcza dodatkowych informacji diagnostycznych
- C. **Proces analizy, identyfikacji i naprawy błędów (tzw. *bugów*) w kodzie źródłowym programu w celu zapewnienia poprawnego działania aplikacji**
- D. Proces, w którym programista umożliwia użytkownikom testowanie oprogramowania w celu znalezienia i zgłaszania błędów

## Pytanie 3

Czym jest *Garbage Collector* w języku Python i jakie jest jego główne zadanie?

- A. Jest to funkcja w Pythonie, która zbiera nieużywane kody źródłowe i przenosi je do archiwum, aby mogły być wykorzystane w przyszłości
- B. Jest to narzędzie do automatycznego generowania dokumentacji kodu, które śledzi wykorzystanie poszczególnych funkcji i modułów w programie
- C. Jest to rodzaj kreatora kodu, który automatycznie generuje kod dla różnych części aplikacji na podstawie analizy i prognozowania potrzeb programistycznych
- D. **Jest to mechanizm automatycznego zarządzania pamięcią w języku Python, który śledzi i usuwa nieużywane obiekty z pamięci w celu zwolnienia zasobów i uniknięcia wycieków pamięci**